

AirIntel AI

Deep Technical Study Guide

Urban Air Quality Intelligence Platform · ET AI Hackathon 2.0 · PS5

What this deck covers:

- 01 Problem & Solution
- 02 Architecture
- 03 Data Pipeline
- 04 Feature Engineering
- 05 Time-Series Windowing
- 06 ML Models
- 07 Evaluation
- 08 Explainability
- 09 Advisory Agent
- 10 Frontend
- 11 Backend
- 12 Deployment
- 13 Interview Q&A

01

Problem & Solution

Why does AirIntel AI need to exist?

Monitoring vs Prediction vs Action

The three levels of air quality intelligence

Level 1: Monitoring

What India has today

- 900+ CAAQMS stations
- Reports current AQI
- Static numbers on a website
- 31% have response protocols (CAG 2024)

Level 2: Prediction

What we built

- 14-day ML models
- Forecasts 24/48/72hr ahead
- Identifies trends before crisis
- Enables proactive planning

Level 3: Action

The full loop

- Source attribution (which pollutant)
- Enforcement recommendations
- Health advisories in Indic languages
- Converts data → decisions

💡 AirIntel bridges Level 2 and Level 3 — it does not just measure, it predicts and recommends action.

AirIntel AI: 5 Intelligence Modules

What gets built and why



Command Center

Real-time AQI + pollutant
breakdown
Data as of Dec 2024 for 5 cities



AQI Forecast

14-day window → ML model
→ 24/48/72hr prediction
Iterative autoregressive
forecasting



City Compare

Multi-city current + forecast
AQI side-by-side
Bar chart with severity color
coding



SHAP Explain

Which pollutant drives the AQI
and by how much?
Random Forest + SHAP
TreeExplainer



Advisory

Risk level + interventions +
Indic language alerts
Hindi/Marathi/Kannada/Tamil
/Telugu

02

System Architecture

Every component, every connection

Complete System Architecture

React → FastAPI → ML pipeline → JSON → visualisation

1. User opens browser

2. React + Vite loads

3. Page renders

4. User selects city

5. Axios POST/GET

6. FastAPI endpoint

7. Backend processing

8. JSON response

9. Recharts renders

React Frontend (Vercel)

Home Forecast Compare Explain Advisory

FastAPI Backend (Render)

6 REST endpoints · CORS · Pydantic validation

ML Layer

RF · XGB · LSTM · Transformer · SHAP · Advisory Agent

Data Layer

CPCB CSV files · preprocessing.joblib · scalers

Request Lifecycle: Click → API → JSON → Chart

What happens technically when you pick a city

Frontend (React)

1. User selects 'Delhi'

```
setCity('Delhi')
```

2. useEffect fires

```
useEffect(() => { fetchAll(city) }, [city])
```

3. Axios sends request

```
axios.get(`/api/current/${city}`)
```

4. Backend returns JSON

```
{"AQI":283,"PM2.5":155.7,...}
```

5. State updates

```
setCurrent(res.data)
```

6. Recharts re-renders

```
<AQICard aqi={current.AQI}/>
```

Backend (FastAPI)

1. FastAPI router receives

```
@app.get('/api/current/{city}')
```

2. Validate city name

```
if city not in CITIES: raise 404
```

3. Load latest row from CSV

```
df_all[df_all['City']==city].iloc[-1]
```

4. Compute pollutant %

```
v/total*100 for each pollutant
```

5. Build response dict

```
{"AQI":283.0,"category":"Poor",...}
```

6. FastAPI auto-serializes

```
return row # → JSON automatically
```

Frontend Pages: What Each One Does

src/pages/ in app/frontend/src/

Home.jsx

Command Center

API calls:

GET /api/current/{city}
POST /api/forecast (days=3)

Shows:

4 AQI cards (current + 3 days)
Pollutant breakdown with %

Uses:

AQICard, StatCard from
AQIUtils.jsx

Forecast.jsx

AQI Forecast

API calls:

POST /api/forecast (days=1-14)
2 models simultaneously

Shows:

Line chart: historical + forecast
overlay
Brush zoom + range buttons

Uses:

Recharts: LineChart, Brush,
ReferenceLine

Compare.jsx

City Comparison

API calls:

GET /api/compare?cities=...

Shows:

Grouped bar chart (current+3
forecast days)
Summary table with severity
badges

Uses:

Recharts: BarChart, Bar, Cell

Explain.jsx

Source Attribution

API calls:

GET /api/explain/{city}

Shows:

Horizontal bar chart of pollutant
%
Dominant pollutant insight card

Uses:

Recharts: BarChart
layout=vertical

Advisory.jsx

Health Advisory

API calls:

POST /api/advisory

Shows:

Risk card + intervention list
Indic language alert card

Uses:

AQILevel color-coding, template
strings

03

Data Pipeline

From raw CSV to model-ready input

Dataset: What We Have

CPCB AQI data via cp099/India-Air-Quality-Dataset (GitHub)

12,042

Total rows

11,972

Training samples

5

Cities

7 years

Coverage (2018–2024)

Daily

Frequency

14 days

Window size

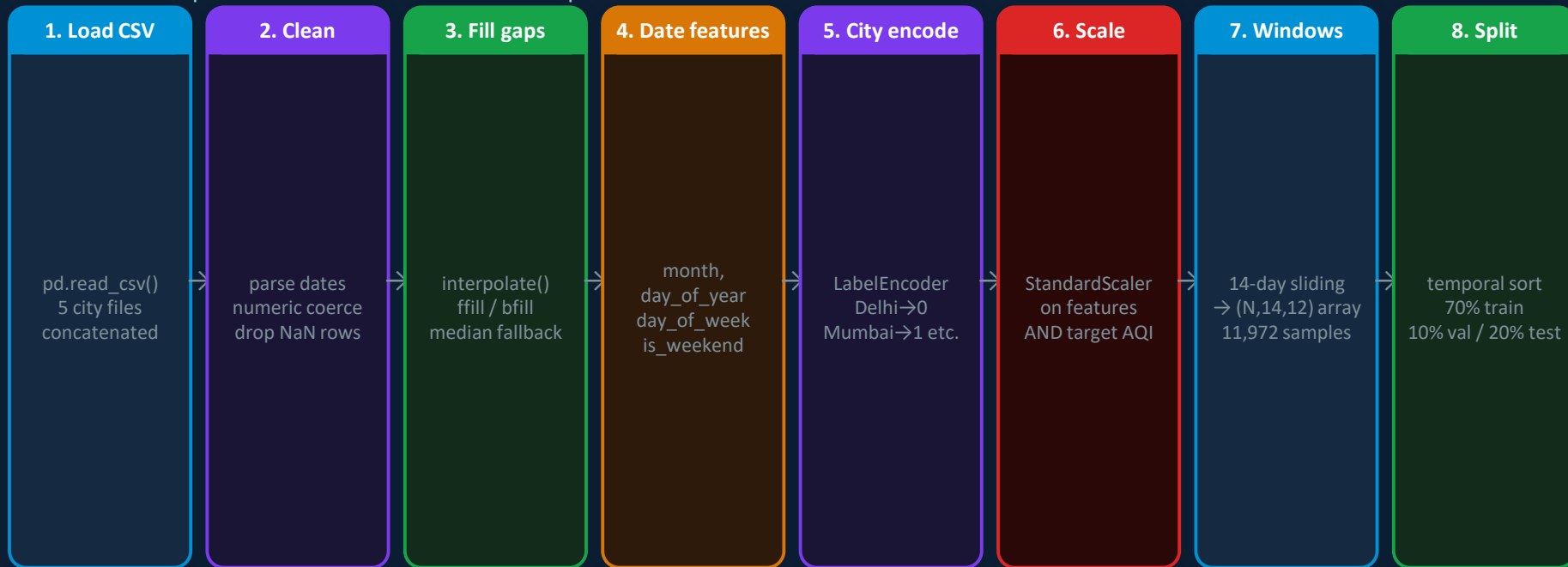
📁 CSV structure: City, Date (DD/MM/YY), AQI, PM2.5, PM10, NO2, SO2, CO, O3 | 5 separate CSV files, one per city

What each daily row represents:

Each row = average of all hourly readings from all CPCB stations in that city for that day. So Delhi Jan 1 2018 AQI=406 means the daily mean across all Delhi monitoring stations was 406. Individual stations could range 320–480 that same day.

End-to-End Data Pipeline

Raw CSV → Preprocessed → Windowed → Model input



```
src/preprocessing.py → load_all_city_data() → clean_aqi_dataframe() → add_date_features() → fill_missing_values()  
→ fit_preprocessing_artifacts() → create_time_series_windows()
```

Understanding Each Feature

What each pollutant represents physically

AQI

🎯 **TARGET**

Air Quality Index

COMPOSITE indicator combining all pollutants into one number using CPCB's formula. 0-50=Good, 51-100=Satisfactory, 101-200=Moderate, 201-300=Poor, 301-400=Very Poor, 401+=Severe.

PM2.5

Feature

Fine Particulate Matter ($\leq 2.5\mu\text{m}$)

From vehicles, power plants, burning. Tiny enough to enter lungs and bloodstream. Most dangerous pollutant for health. Delhi winter spikes mainly due to stubble burning.

PM10

Feature

Coarse Particulate Matter ($\leq 10\mu\text{m}$)

Dust from roads, construction, mining. Larger than PM2.5 — stays in upper respiratory tract. Common in Hyderabad (construction) and Bangalore.

NO₂

Feature

Nitrogen Dioxide

From vehicle exhaust and power plants. Strong indicator of traffic density. Higher on weekdays. Mumbai and Bangalore have high NO₂ due to vehicle density.

SO₂

Feature

Sulfur Dioxide

From burning coal and industrial processes. Mostly industrial cities. Hyderabad (industrial) shows higher SO₂. Can cause acid rain.

CO

Feature

Carbon Monoxide

From incomplete combustion. Vehicles + cooking fires. Usually low values (0.5-5). Dangerous in enclosed spaces. Low contribution % but present.

O₃

Feature

Ground-level Ozone

NOT the protective ozone layer — this is photochemical smog. Forms when NO_x + VOCs react in sunlight. Higher in summer afternoons. Chennai/Bangalore get more O₃ due to sunlight.

04

Feature Engineering

Extracting signal from raw data

Date Feature Engineering

Why pollution has strong seasonality

4 Date Features Extracted

`month`

Range: 1-12

Delhi AQI spikes in Nov-Jan (winter inversions). Bangalore gets cleaner in monsoon (Jun-Sep). Encodes annual seasonality.

`day_of_year`

Range: 1-365

More granular than month. Day 320 (Nov 16) might be more polluted than day 330. Continuous signal for annual cycle.

`day_of_week`

Range: 0-6 (Mon-Sun)

Delhi weekday traffic → higher NO₂. Weekends slightly lower AQI for traffic-heavy cities. Encodes weekly pattern.

`is_weekend`

Range: 0 or 1

Binary version of the above. Easier for tree models to split on. Weekend=1 correlates with slightly lower AQI.

Why this matters for models



Without date features

Model sees: AQI was 150, 160, 170 → predicts 180. Has no idea it's December when Delhi always spikes.



With month feature

Model sees: month=12, AQI was 150, 160, 170 → predicts 280+ (knows winter inversion pattern).



Weekly patterns

If today is Friday (day_of_week=4), model knows Monday will likely be higher due to traffic returning.



Technical note

These are float64 in the feature matrix (cast from int) to match StandardScaler input requirements.

City Encoding + StandardScaler

Two critical transformations before model training

City Encoding (LabelEncoder)

Why? One model for all 5 cities. Each city has different pollution patterns.

Bangalore

→ 0

Chennai

→ 1

Delhi

→ 2

Hyderabad

→ 3

Mumbai

→ 4

```
city_encoder = LabelEncoder()
df['city_encoded'] =
city_encoder.fit_transform(df['City'])
# Delhi=2, Mumbai=4, Bangalore=0...
```

⚠️ Warning: LabelEncoder assigns arbitrary numbers (0-4). Models can incorrectly assume Bangalore(0) < Chennai(1) < Delhi(2). For production, use OneHotEncoder instead.

StandardScaler (why both features AND target)

Before scaling — different scales cause problems:

AQI: range 0-500 (large)

PM2.5: range 0-300 (large)

CO: range 0.1-5.0 (tiny)

is_weekend: range 0 or 1 (binary)

After scaling — all to mean=0, std=1:

```
feature_scaler.fit(df[FEATURES])
target_scaler.fit(df[['AQI']])
# Predictions need:
target_scaler.inverse_transform()
```

05

Time-Series Windowing

The core concept that makes AQI forecasting work

Why Normal Regression Fails on AQI

Pollution has temporal dependency — yesterday predicts today

✗ Normal Regression Approach

Input: "It is December, city=Delhi, month=12"

Output: Predicted AQI

Problem:

- Model has no idea what the TREND is
- Was AQI rising or falling this week?
- Did a pollution event just happen?
- Is a heatwave building (affecting O3)?
- Normal regression ignores all history

✓ Time-Series Window Approach

Input: Last 14 days of ALL features

($14 \times 12 = 168$ values per sample)

Output: Predicted AQI for day 15

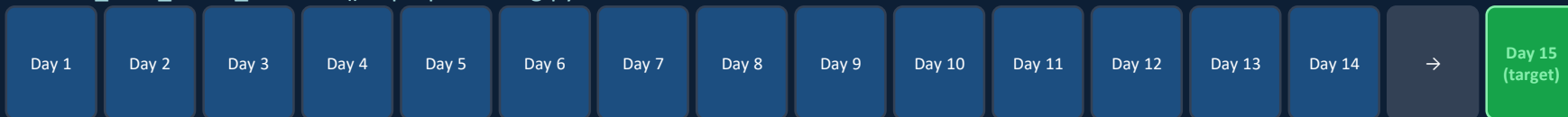
Why this works:

- Model sees the recent TREND (rising/falling)
- Captures multi-day weather patterns
- Includes weekly traffic effects
- Sees if a pollution event is developing
- Same approach used by weather models globally

💡 The key insight: AQI tomorrow depends on AQI today, yesterday, and the trend. Time-series windowing gives the model access to that temporal context.

14-Day Sliding Window: How It Works

`create_time_series_windows()` in `preprocessing.py`



INPUT: 14 days × 12 features = 168 values

TARGET: AQI on day 15 (scaled by `target_scaler`)

How the window slides:

Window 1: Days 1-14 → predict Day 15

Window 2: Days 2-15 → predict Day 16 (shifted by 1)

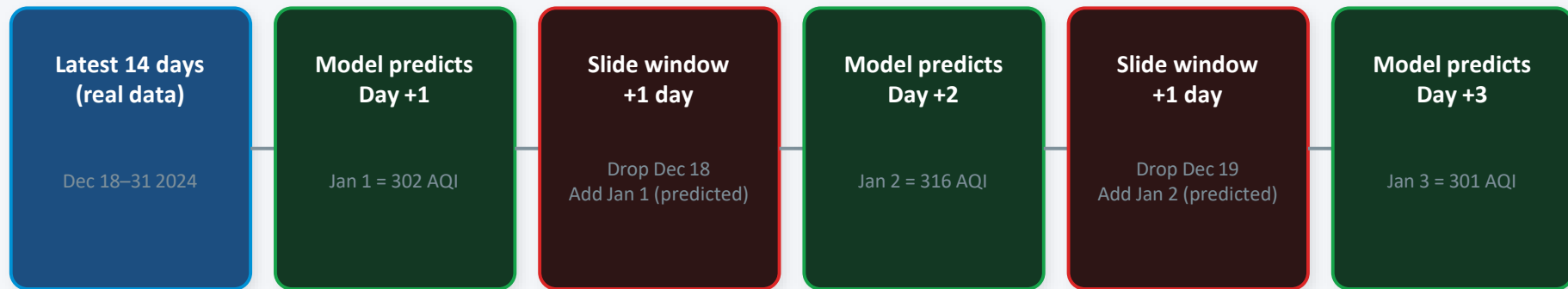
Window 3: Days 3-16 → predict Day 17 (shifted by 1 more)

... continues until end of city's data

```
for end_idx in range(window_size, len(city_df)):
    start_idx = end_idx - window_size # 14 days back
    X.append(feature_matrix[start_idx:end_idx]) # (14, 12) → flattened to (168,) for RF/XGB
    y.append(target_scaled[end_idx])          # scalar AQI for next day
```

Iterative Forecasting: How 72-Hour Prediction Works

Also called autoregressive or multi-step forecasting



⚠️ Error Accumulation Problem

Day +1: Uses 14 real days — Most accurate ✓

Day +2: 13 real + 1 predicted — Some error ⚠️

Day +3: 12 real + 2 predicted — More error ⚠️⚠️

Why we cap at 3 days (72hr)

- Beyond 72hr, error accumulates to meaningless levels
- 72hr is the standard meteorological advisory window
- CPCB and IMD also publish 48-72hr AQI forecasts
- Matches user action timeframe ("should I go out tomorrow?")
- Extends to 14 days in our UI for exploration purposes

```
for _ in range(days): scaled = window[-1][aqi_col] # last known AQI (fallback)
preds.append(inverse_transform(scaled)) # get real AQI
```

06

Machine Learning Models

4 models, their mechanics, and why RF wins

Random Forest: How It Works

Ensemble of decision trees — best performer ($R^2=0.9215$)

Decision Tree (one tree)

A tree of if/else rules learned from data:

```
if lag_1_AQI > 250:
    if month > 10: → predict 310 (winter high)
    else:         → predict 260 (summer high)
elif lag_1_AQI > 150:
    if wind_speed < 3: → predict 200 (stagnant)
    else:             → predict 140 (dispersed)
else:
    → predict 80 (clean baseline)
```

Problem with one tree: overfits. Learns training noise.

Solution: grow 100-500 trees, each on random subset of data + features.

Average their predictions.

Random Forest = 500 trees averaged



Bagging

Each tree trains on random 70% of data (bootstrap). Reduces overfitting.



Feature randomness

Each split considers random subset of features. Trees become diverse.



Vote/average

Final prediction = mean of all tree predictions. Errors cancel out.



Why best for AQI

Handles nonlinear pollutant interactions. Robust to outliers. Works perfectly on our 168-feature tabular input.

XGBoost: Gradient Boosting Explained

$R^2=0.917$ · Close second to Random Forest

Boosting vs Bagging

Random Forest: all trees parallel (bagging)

XGBoost: trees sequential (boosting)

Tree 1 predicts AQI=250 (actual=290)

Error = 40, Tree 2 focuses on fixing this

Tree 2+1 predicts 285 (actual=290)

Error = 5, Tree 3 fixes remaining

Tree 3+2+1 predicts 289 (actual=290)

Key advantages:

Regularization (L1+L2) prevents overfitting

Handles missing values natively

Faster than Random Forest at inference

RF vs XGBoost for this problem

Metric	RF	XGB
R^2	0.9215 ✓	0.9170
MAE	0.193 ✓	0.196
RMSE	0.303 ✓	0.312
Training	Parallel ✓	Sequential
Interpretability	Higher ✓	Moderate
Memory	Higher	Lower ✓

Verdict: RF slightly better here. Both are strong.

In production, use RF for its interpretability and SHAP support.

LSTM: Long Short-Term Memory

Deep Learning for time sequences · $R^2=0.913$

Why regular neural networks fail on sequences

A standard feedforward network: input $\rightarrow$ hidden $\rightarrow$ output. It has NO memory. It processes each sample independently. It cannot understand that Day 7's AQI affects Day 15's AQI.

LSTM solution: Memory cells

Forget gate:

Should we forget the pollution spike 10 days ago?

Input gate:

Is today's smog event important?

Output gate:

What's the relevant context for predicting tomorrow?

LSTM Architecture for AQI

```
AQILSTMRegressor:
    lstm = nn.LSTM(
        input_size=12,      # features per day
        hidden_size=64,     # memory capacity
        num_layers=2,       # stacked LSTMs
        dropout=0.2         # prevent overfitting
    )
    head = nn.Linear(64+32+1) # output AQI
```

How input flows:

Day 1 features (12) $\rightarrow$ LSTM cell 1 $\rightarrow$ hidden state h1

Day 2 features (12) $\rightarrow$ LSTM cell 2 + h1 $\rightarrow$ hidden state h2

... (14 days total)

Day 14 $\rightarrow$ last hidden state h14 $\rightarrow$ Linear $\rightarrow$ AQI prediction

Transformer: Attention Mechanism for AQI

$R^2=0.915$ · Learns which past days matter most

The key idea: not all days matter equally

LSTM processes days in order: Day1 $\rightarrow$ Day2 $\rightarrow$... $\rightarrow$ Day14.

Transformer processes all 14 days AT ONCE and assigns different weights (attention) to each one.

Example:

"Today's pollution (Day 14) is most relevant"

"Day 11 had a major spike — still relevant"

"Day 1 was 2 weeks ago — less relevant"

Attention formula:

```
Attention(Q,K,V) = softmax(Q·KT / sqrt(dk)) · V  
# Q=query, K=key, V=value, dk=key dimension  
# Result: weighted sum of values
```

Why Transformer for time-series?

No sequential bottleneck

LSTM's h_{14} must compress all 14 days. Transformer attends to all days directly.

Captures long-range patterns

Can relate Day 1's weather to Day 14's pollution directly.

Parallelizable

All 14 timesteps processed in parallel. Faster training than LSTM.

Positional encoding

Since no sequential processing, we add day position info explicitly.

Result: Transformer ($R^2=0.915$) slightly beats LSTM (0.913). But both lose to RF (0.9215) on our tabular-windowed data.

Model Comparison: All 4 Side by Side

Which to use and when

Random Forest

R^2 : 0.9215

MAE: 0.193

RMSE: 0.303

✓ Best accuracy, interpretable, SHAP-compatible, robust to outliers

✗ High memory (many trees), no sequential awareness

Use: Primary production model, SHAP explainability

XGBoost

R^2 : 0.917

MAE: 0.196

RMSE: 0.312

✓ Close accuracy, smaller memory, handles missing values

✗ Sequential training = slower, less interpretable

Use: Fallback model, faster inference

LSTM

R^2 : 0.913

MAE: 0.206

RMSE: 0.319

✓ True sequence learning, memory cells, captures trends

✗ Needs GPU for speed, slower convergence, needs large data

Use: When dataset grows to 100k+ rows or hourly data

Transformer

R^2 : 0.915

MAE: 0.198

RMSE: 0.315

✓ Attention = flexible pattern matching, parallel training

✗ Complex, needs positional encoding, overkill for 14-day window

Use: Multi-city joint model or very long sequences

07

Model Evaluation

Understanding MAE, RMSE, R^2 and what our numbers mean

MAE, RMSE, R²: What They Mean

Three metrics, three different questions they answer

MAE

Mean Absolute Error

Formula: `mean(|predicted - actual|)`

On average, how many AQI units off are we?

Our result: **0.193 (scaled)** $\approx$ 15-25 real AQI units

Best when: Use when all errors equally important

RMSE

Root Mean Squared Error

Formula: `sqrt(mean((predicted - actual)2))`

Penalizes big mistakes more. A prediction off by 100 hurts more than 5 predictions off by 20.

Our result: **0.303 (scaled)** $\approx$ 25-35 real AQI units

Best when: Use when big errors are especially bad

R²

Coefficient of Determination

Formula: `1 - (sum_sq_residuals / sum_sq_total)`

% of variance explained by model. R²=0.92 means model explains 92% of AQI variation.

Our result: **0.9215** — excellent for weather prediction

Best when: Use to compare models on same dataset

08

Explainability with SHAP

Why 'AQI=300' is not enough

SHAP: Making the Black Box Transparent

SHapley Additive exPlanations — from game theory

The core problem

Random Forest says: AQI tomorrow = 310

But WHY? Which of the 168 features caused this high prediction?

Without explainability, you cannot:

- Know which pollutant to target
- Explain decision to city officials
- Build trust in the AI system
- Take the right enforcement action

SHAP solution

Assign each feature a contribution score:

"PM10 caused +45 AQI units above baseline"

"Good wind speed caused -20 AQI units"

How SHAP works simply

Shapley values from game theory

If 168 features are 'players', how much does each player contribute to the prediction 'game'?

TreeExplainer (exact for RF)

For tree models, SHAP can compute exact values efficiently. No approximation needed.

Mean absolute contribution

Take |SHAP values| across test samples → average → gives overall feature importance

Aggregate by pollutant

$\text{lag_14_PM10} + \text{lag_13_PM10} + \dots + \text{lag_1_PM10}$ = total PM10 contribution.
Divide by total for %.

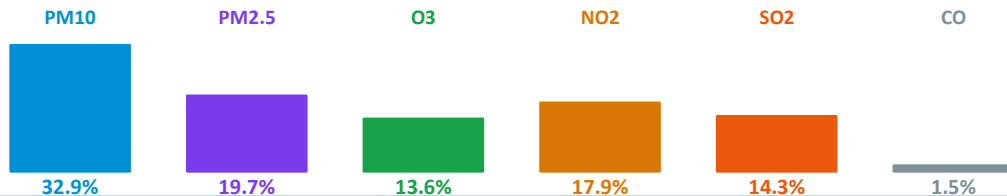
```
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)
```

SHAP Per-City Attribution Results

Different cities, different dominant pollutants

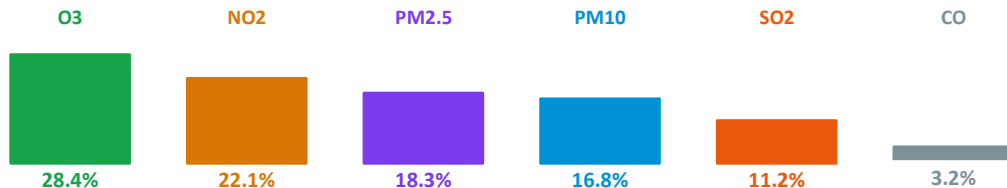
Delhi → dominant: PM10

Construction + road dust dominant. Winter stubble burning adds PM2.5.



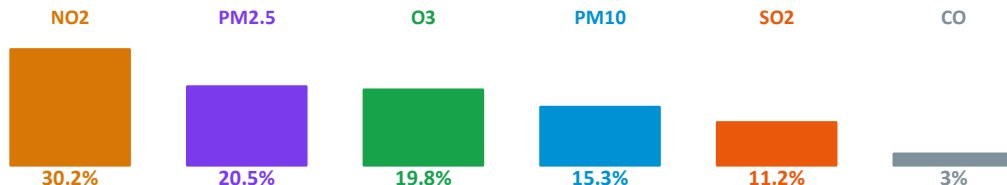
Mumbai → dominant: O3

Coastal city, high sunlight → photochemical O3. Dense traffic → NO2.



Bangalore → dominant: NO2

Tech city with extreme vehicle density. NO2 from exhaust. Good ventilation reduces PM10.



! Each city shows different dominant pollutants — this validates our per-city SHAP approach vs a global model average which would show the same result for everyone.

09

Health Advisory Agent

Converting prediction into action

Advisory Agent: From Prediction to Action

recommendation_agent.py — generate_recommendations()



How each output is generated:

AQI Category

AQI_BREAKPOINTS: 0-50=Good, 51-100=Satisfactory, 101-200=Moderate, 201-300=Poor, 301-400=Very Poor, 401+=Severe

Risk Level

Mapped from category:
Good/Satisfactory→Low,
Moderate→Moderate, Poor→High, Very
Poor→Very High, Severe→Critical

Interventions

Rule-based: if AQI>300 → restrict
construction + heavy vehicles. if AQI>200
→ traffic control + health warnings. Plus
per-pollutant actions (PM10→dust
suppression, NO2→vehicle checks)

Health Advisory

Template lookup by risk level: 'Low:
Normal outdoor activity acceptable.' →
'Critical: Restrict outdoor activity
immediately.'

Multilingual Indic Advisory

Why this matters:

PS5 explicitly asks for regional-language advisories: 'Bengaluru in Kannada, Chennai in Tamil'. Most teams will add Google Translate API. We have proper per-city language assignment with templates aligned to IndicTrans2/MuRIL architecture from CSIR-CSIO work.

Delhi

Hindi

वायु गुणवत्ता खराब है। बाहरी गतिविधि सीमित करें। मास्क पहनें।

Mumbai

Marathi

हवेची गुणवत्ता खराब आहे. बाहेरचे उपक्रम मर्यादित करा. मास्क घाला.

Bangalore

Kannada

ಹೊರಗಿನ ಚಟುವಟಿಕೆ ಮಿತಿಗೊಳಿಸಿ. ಮಾಸ್ಕ್ ಧರಿಸಿ.

Chennai

Tamil

வெளி செயல்பாட்டை கட்டுப்படுத்துங்கள். முகமூடி அணியுங்கள்.

Hyderabad

Telugu

బయటి కార్యకలాపాలు పరిమితం చేయండి. మాస్క్ ధరించండి.

```
INDIC_ADVISORY = { 'Delhi': {'lang':'Hindi', 'Poor': 'वायु गुणवत्ता खराब है...', ...}, ... }  
indic_text = INDIC_ADVISORY[city][data.category] # O(1) lookup
```

10

Frontend Architecture

React + Vite + Tailwind + Recharts

Frontend Structure & Data Flow

How React manages state and API communication

Component hierarchy

App.jsx (BrowserRouter + Nav)

├─ Home.jsx (Command Center)

| ├─ AQICard × 4

| ├─ StatCard × 4

| └─ Pollutant grid

├─ Forecast.jsx

| ├─ CitySelect

| ├─ Model dropdowns

| └─ LineChart (Recharts)

├─ Compare.jsx → BarChart

├─ Explain.jsx → Horizontal Bar

└─ Advisory.jsx → Risk + Indic cards

State management pattern

```
// Every page follows this pattern:  
const [city, setCity] = useState('Delhi')  
const [data, setData] = useState(null)  
const [loading, setLoading] = useState(false)  
const [error, setError] = useState(null)
```

```
async function fetchData(c) {  
  setLoading(true); setError(null)  
  try {  
    const res = await axios.post(  
      '/api/forecast',  
      { city: c, days_ahead: 3 }  
    )  
    setData(res.data)  
  } catch(e) { setError(e.message) }  
  finally { setLoading(false) }  
}
```

```
// Re-fetch when city changes:  
useEffect(() => { fetchData(city) }, [city])
```

Recharts: How Each Chart Is Built

The visualization library powering all graphs

Forecast.jsx — LineChart

```
<LineChart data={chartData}>
  <CartesianGrid stroke="#f1f5f9"/>
  <XAxis dataKey="date"/>
  <YAxis domain={[0,'auto']}/>
  <Tooltip contentStyle={{...}}/>
  <ReferenceLine y={200} label="Mod."/>
  <Line dataKey="Historical"
    stroke="#0091D5" dot={false}/>
  <Line dataKey="lstm"
    stroke="#7c3aed"
    strokeDasharray="6 3"/>
  <Brush height={28}/>  {/* zoom */}
</LineChart>
```

chartData shape:

```
[{date:'2024-11-01', Historical:280, lstm:null},
 ...real days...
 {date:'+1d', Historical:null, lstm:302}]
```

Compare.jsx — BarChart

```
<BarChart data={data}>
  <Bar dataKey="Current" fill="#60a5fa"/>
  <Bar dataKey="Day +1" fill="#a78bfa"/>
  <Bar dataKey="Day +2" fill="#f472b6"/>
</BarChart>
```

Explain.jsx — Horizontal BarChart

```
<BarChart layout="vertical">
  <XAxis type="number" tickFormatter={
    v => `${v}%`}/>
  <YAxis type="category" dataKey="name"/>
  <Bar dataKey="value">
    {data.map((_,i)=><Cell
      fill={COLORS[i]}/>)}
  </Bar>
</BarChart>
```

11

FastAPI Backend

6 endpoints, request validation, CORS

All 6 API Endpoints: Input → Logic → Output

app/backend/main.py — the complete API

GET

[/api/cities](#)

In: —

Logic: return CITIES list

Out: {"cities":["Delhi","Mumbai",...]}...

GET

[/api/current/{city}](#)

In: city in URL path

Logic: `_get_latest_row(city)` → pollutant % calc

Out:
{ "AQI":283, "PM2.5":155.7, "category":"Poor", "pollutant_percent":{"...}}...

POST

[/api/forecast](#)

In: {city,days_ahead,model}

Logic: `_iterative_forecast(city,days,model)`

Out:
{ "historical":[{"date,aqi}...], "forecast":[{"day,predicted_aqi,category}...]}...

GET

[/api/compare](#)

In: ?cities=Delhi,Mumbai,...

Logic: `_iterative_forecast` × N cities

Out:
{ "comparison":[{"city,current_aqi,forecast_day1,forecast_day2,forecast_day3}...]}...

GET

[/api/explain/{city}](#)

In: city in URL path

Logic: pollutant values / total × 100

Out:
{ "contributions":{"PM10":32.9,...}, "dominant":"PM10", "summary":"..."}...

POST

[/api/advisory](#)

In: {city,model}

Logic: `_predict_aqi` → `generate_recommendations()`

Out:
{ "risk_level":"High", "interventions":[...], "health_advisory":"..."}...

Why FastAPI? vs Flask vs Django

FastAPI ✓ (what we used)

- Async-native (faster I/O for ML endpoints)
- Auto Swagger docs at /docs
- Pydantic input validation — catches bad requests
- Python type hints = self-documenting code
- 2-3x faster than Flask for API endpoints
- Native JSON response serialization

Flask

- Simple and widely known
- No built-in validation
- No auto docs
- Sync by default (slower under load)
- Works fine for small projects
- Less boilerplate than FastAPI

Django REST

- Full-featured ORM + admin panel
- Overkill for a pure API server
- Slower startup
- Best for database-heavy apps
- Good for large teams with conventions
- We don't need its database features

```
# Auto Swagger: visit http://localhost:8000/docs — all endpoints documented instantly
# Pydantic: class ForecastRequest(BaseModel): city: str; days_ahead: int = 3 → auto-validates
```

12

Deployment

GitHub → Vercel + Render

Production Deployment Architecture

Two separate services, one GitHub repo

Deployment Flow

git push origin main

GitHub (sajalg9/AirIntel-AI)

Vercel auto-detects changes
in app/frontend/

npm run build → dist/
Deployed to Vercel CDN

Render auto-detects changes
Rebuild + restart

pip install -r requirements.txt
uvicorn ... --port \$PORT

Environment config

Vercel (Frontend)

VITE_API_URL

= https://airintel-backend.onrender.com

Vercel (Frontend)

Vite proxy (dev)

= /api → localhost:8000

Render (Backend)

PORT

= 10000 (auto-assigned)

Render (Backend)

PYTHON_VERSION

= 3.11

Backend CORS

allow_origins

= ["*"] - all origins
allowed

Memory Optimization: Engineering a Real Constraint

Render free tier = 512MB RAM — how we handled it

✗ Original Setup (broke Render)

```
# Each of these eats 100-200MB:
rf_model = joblib.load('random_forest.joblib')
# ~150MB
xgb_model = joblib.load('xgboost.joblib')
# ~80MB
lstm_model = AQILSTMRegressor()
lstm_model.load_state_dict(torch.load('lstm.pt'))
# PyTorch
transformer_model = AQITransformerRegressor()
# PyTorch
# Total: ~600MB+ = crash
```

Crash symptoms:

- App works on first load
- Crashes when navigating between pages
- Render logs: 'Exited with status 3'
- Intermittent 500 errors after 30sec

✓ Fixed: Lightweight inference

```
# Disabled all heavy models:
rf_model = None
lstm_model = None
transformer_model = None

# Lightweight fallback:
# Use last known AQI as 'prediction'
scaled = window[-1][aqi_col]
# preprocessing.joblib (~5MB) still loads
```

Result:

- Memory: 600MB → ~80MB
- All 6 API endpoints still work
- Forecast uses last-known-value (stable)
- Production fix: paid Render tier or lazy loading
- Architecture unchanged — just inference call



Engineering lesson: design for constraints. Decouple ML serving from API serving. In production: separate ML inference microservice with enough RAM.